LD9-V1 Bounded Tiled Contour Extraction Specification

Exact crossing-cell planning, deterministic periodic tile ownership, indexed marching-cubes geometry, and bounded transient memory

mdstats development specification

2026-07-21

Contents

LD9-V1 - Bounded tiled contour extraction

Package target: `mdstats 0.19.59a0`
Status: implemented; tiled extraction is the normal nonwinding local-sparse mesh path
Primary modules:

```
mdstats.plotting.density_contour_tiles
mdstats.plotting.density_tiled_mesh
mdstats.plotting.density_sparse_mesh
```

Purpose

LD9-V1 removes the pathological one-marching-cubes-call-per-logical-cell renderer while preserving the scientific density field and its highest-density-region (HDR) threshold exactly. The stage partitions exact contour-crossing cells into bounded render tiles, extracts each tile with one Lewiner marching-cubes call, assigns deterministic logical-edge identities to vertices, and releases tile-local scalar and raw geometry before processing the next tile.

The stage changes only contour extraction and mesh assembly. It does **not**:

- alter the density grid, Gaussian bandwidth, 10^{-8} kernel-tail cutoff, CIC operator, normalization, or HDR threshold;
- simplify the indexed mesh;
- relax the final browser face, vertex, trace, or HTML limits;
- authorize the interactive-browser production gate.

Mesh simplification and scene-wide browser budgeting remain LD9-V2 and LD9-V3 responsibilities.

External methods and mdstats-specific adaptations

Regular-grid isosurface extraction follows the marching-cubes construction introduced by Lorensen and Cline (1987). Ambiguous cases are resolved by the topologically consistent Lewiner et al. (2003) implementation supplied by scikit-image. The following are mdstats-specific:

- periodic logical-cell ownership;
- render-tile partitioning and positive-face node halos;
- logical-grid-edge vertex keys;
- canonical seam-plane copies at fractional coordinates zero and one;
- exact high-node contour-candidate pruning;
- transient-resource contracts and transactional failure;
- deterministic cross-tile indexing and canonical ordering.

Scientific invariants

For scalar field ρ and scientific HDR threshold t_q, V1 preserves:

$$\rho, \qquad t_q, \qquad \int_{\rho \geq t_q} \rho \, dV,$$

and the logical crossing-cell set

$$\mathcal{C}_q = \left\{ \mathbf{c} : \min_{\mathbf{v} \in V(\mathbf{c})} \rho(\mathbf{v}) < t_q^{(32)} < \max_{\mathbf{v} \in V(\mathbf{c})} \rho(\mathbf{v}) \right\},$$

where $t_q^{(32)}$ is the interior `float32` rendering level derived from the immutable scientific threshold.

A crossing cell necessarily has at least one corner strictly above $t_q^{(32)}$. Candidate planning therefore begins only from stored nodes satisfying

$$\rho_i > t_q^{(32)},$$

rather than from every positive node in the truncated Gaussian support. This pruning is exact; it removes Gaussian-tail nodes that cannot seed a crossing cell.

scikit-image evaluates the contour interpolation in single precision. If the rounded scientific threshold lies too close to the field maximum, every interpolated vertex of a point-like CIC field can collapse numerically onto the maximal node. V1 therefore constrains only the display level to lie at least 16 `float32` ULPs below the rounded maximum in this endpoint-degenerate case. The scientific threshold remains unchanged and is serialized separately. This guard is deterministic and does not alter ordinary interior thresholds.

Public contracts

Extraction options

```python
@dataclass(frozen=True, slots=True)
class MeshExtractionOptions:
    render_tile_shape: tuple[int, int, int] = (32, 32, 32)
    max_crossing_cells_per_tile: int = 131_072
    max_raw_faces_per_tile: int = 655_360
    max_raw_vertices_per_tile: int = 1_572_864
    max_transient_mesh_bytes: int = 256 * 1024**2
    max_total_crossing_cells: int = 4_000_000
    max_total_raw_faces: int = 20_000_000
    max_total_raw_vertices: int = 60_000_000
    max_planning_workspace_bytes: int = 512 * 1024**2
```

Storage blocks, LD8 convolution tiles, and LD9 render tiles are independent concepts. Changing `render_tile_shape` may change work partitioning but must not change the canonical indexed result on validated nonambiguous fixtures.

Tile plan

```
ContourTilePlan(
    field_key=...,
    grid_shape=...,
    render_level=...,
    render_tile_shape=...,
    tiles=...,
    total_crossing_cell_count=...,
    total_raw_face_upper_bound=...,
    total_raw_vertex_upper_bound=...,
    maximum_tile_transient_bytes=...,
    planning_bytes=...,
)
```

Each `ContourRenderTile` owns a disjoint half-open logical-cell box

$$[c_x^0, c_x^1) \times [c_y^0, c_y^1) \times [c_z^0, c_z^1),$$

and its scalar brick includes one positive-face node halo. Terminal tiles may be smaller than the nominal tile shape.

Extraction result

```
TiledMeshExtractionResult(
    vertices_fractional=...,
    vertices_cartesian=...,
    faces=...,
    tile_reports=...,
    raw_vertex_count=...,
    raw_face_count=...,
    maximum_tile_transient_bytes=...,
    retained_geometry_bytes=...,
    estimated_peak_bytes=...,
)
```

Canonical JSON round trips require `include_geometry=True`; summary-only payloads are deliberately non-restorable.

Planning algorithm

1. Resolve the scientific HDR threshold and interior `float32` render level.
2. Collect only stored nodes strictly above the render level.
3. Expand each selected node to its at most eight adjacent periodic logical cells.
4. Deduplicate flat cell indices.
5. Gather all eight scalar corners in bounded batches.
6. Retain exactly the cells crossing the render level.
7. Assign each crossing cell to one render tile by integer division of its canonical logical-cell coordinate.
8. Sort tiles lexicographically and record exact crossing counts.
9. Preflight per-tile and global raw-geometry upper bounds before marching cubes.

For N_c crossing cells, the standard marching-cubes table permits no more than five triangles per cell. V1 therefore uses the conservative bound

$$N_{f,\mathrm{raw}} \leq 5N_c.$$

The vertex upper bound is likewise conservative and is used only for allocation rejection, not scientific reasoning.

Tile extraction

For every planned tile:

1. allocate one dense scalar brick of shape $(n_x + 1, n_y + 1, n_z + 1)$;
2. gather scalar values from the sparse scientific field, with periodic node wrapping;
3. call `skimage.measure.marching_cubes` once at the fixed render level;
4. classify each triangle as wholly inside, wholly outside, or intersecting the canonical cell boundary;

5. retain inside triangles directly, discard outside triangles, and clip only intersecting triangles;
6. map vertices to deterministic logical-grid-edge keys;
7. append indexed faces to the retained global builder;
8. release scalar and raw tile arrays before advancing.

No dense global scalar field is allocated.

Deterministic vertex ownership

An uncut marching-cubes vertex lies on one logical grid edge and is identified by a key containing:

- the lifted logical lower endpoint;
- the edge axis;
- the canonical seam copy when the edge lies on a periodic cut.

The final coordinate is recomputed from the two scientific endpoint values and the common render level, not copied from whichever tile encountered the edge first. Consequently, adjacent tiles share one indexed vertex and tile traversal does not introduce floating-coordinate drift.

Vertices on fractional planes zero and one remain distinct canonical-cell copies. Periodic seam validation pairs them geometrically after translation by the corresponding lattice vector.

Resource accounting

Per tile, V1 records:

- crossing cells;
- scalar nodes;
- marching-cubes calls;
- raw vertices and faces;
- newly retained indexed vertices and faces;
- inside, outside, and clipped-triangle counts;
- estimated transient bytes.

The extraction peak estimate is

$$B_{\text{peak}} = B_{\text{retained indexed geometry}} + \max_k B_{\text{transient},k},$$

not the sum of all tile-transient peaks. Candidate and tile-planning workspaces are accounted separately by the outer sparse-mesh resource record.

V1 must fail before extraction when any declared raw or transient bound is exceeded. It must not lower scientific resolution, change the threshold, remove components, or switch to a less accurate contour.

Integration and compatibility

`prepare_sparse_density_mesh(..., extraction_method="tiled")` is the normal nonwinding path. The previous per-cell implementation remains available as

```
extraction_method="legacy_cell"
```

for migration comparison only.

Winding shells retain the established dense-canonical or node-cloud fallback policy. V1 does not change winding classification.

The output still passes the strict existing checks for:

- interior edge incidence;
- periodic boundary-edge pairing;
- maximum seam mismatch;
- logical-cell edge-length bounds;
- duplicate and degenerate triangles.

Strict whole-mesh topology validation remains intentionally enabled in this stage. Its runtime is measured separately and may be optimized in later LD9 work without weakening acceptance semantics.

Focused validation requirements

The implementation is accepted when tests establish:

- options, plans, reports, and geometry-bearing extraction results round-trip through canonical JSON;
- one marching-cubes invocation occurs per nonempty render tile;
- tile boxes are disjoint and cover every crossing cell exactly once;
- partial terminal tiles are valid;
- tile-shape changes preserve canonical geometry on deterministic fixtures;
- tiled and legacy meshes agree geometrically within the declared floating tolerance;
- face-, edge-, and corner-crossing clouds have paired periodic seams;
- raw per-tile failures occur before marching cubes;
- no tile-local raw arrays are retained after tile completion;
- final face limits remain hard;
- package exports and backward-compatible legacy selection work.

Recorded stress evidence

The saved full-resolution four-species stress fields from the original 1,500-frame scene were evaluated at their 50% HDR shells. The scientific fields and thresholds were reused unchanged.

Species	Crossing cells	Render tiles / MC calls	Indexed faces	Indexed vertices	Mesh-preparation time
Na	45,293	110	90,306	45,505	9.30 s
Si	39,064	111	77,948	39,028	8.17 s
Al	42,365	88	84,552	42,334	8.15 s
O	156,809	385	312,668	157,536	29.80 s
Total	**283,531**	**694**	**565,474**	**284,403**	**55.42 s**

A cell-wise implementation would issue one marching-cubes call for each crossing cell. V1 therefore reduces the call count for these shells by approximately

$$\frac{283531}{694} \approx 408.5.$$

On the Na 50% shell, the complete normal mesh-preparation path decreased from 21.69 s with the legacy cell-wise oracle to 9.30 s with V1, a 2.33-fold improvement. The remaining face count is intentionally unsimplified and still exceeds the final scene-wide browser budget when all shells are combined. This is expected: V1 bounds extraction; LD9-V2 must reduce geometry under explicit fidelity constraints.

The larger-shell timing audit also shows that periodic component labeling and strict full-mesh topology validation become significant after marching-cubes call overhead is removed. These steps remain correct and enabled; later stages may accelerate them only while preserving their invariants.

Failure policy

The following raise structured package exceptions rather than producing partial geometry:

- plan/field identity mismatch;
- render-level mismatch;
- invalid or overlapping tile ownership;
- raw per-tile or global resource-limit excess;
- transient workspace excess;
- inconsistent coordinates for a shared logical-edge key;

- nonmanifold interior edges;
- unpaired periodic seams;
- final face-limit excess.

No oversized interactive HTML is authorized by V1. Browser serialization remains governed by the hard LD9-V0/V3 contracts.

Deferred work

LD9-V2 owns:

- bounded tile-local presimplification;
- global periodic seam-aware simplification;
- scalar-field residual, surface-distance, normal, topology, and seam fidelity gates.

LD9-V3 owns:

- post-replication scene-wide face and vertex allocation;
- compact Plotly dtypes and metadata;
- trajectory trace grouping;
- final HTML-byte enforcement.

LD9-V4 owns automated browser acceptance and production-default authorization.

References

1. W. E. Lorensen and H. E. Cline, "Marching Cubes: A High Resolution 3D Surface Construction Algorithm," *ACM SIGGRAPH Computer Graphics* **21**(4), 163–169 (1987). DOI: 10.1145/37402.37422.
2. T. Lewiner, H. Lopes, A. W. Vieira, and G. Tavares, "Efficient Implementation of Marching Cubes' Cases with Topological Guarantees," *Journal of Graphics Tools* **8**(2), 1–15 (2003). DOI: 10.1080/10867651.2003.10487582.